

Syrudas AI: A Local-First AI Workspace with Pluggable Model Providers

Version 0.7.1 · July 2026 · Len · MIT License

Abstract

Syrudas AI is a self-hosted AI workspace for Windows that runs entirely on the user's own machine. It provides streaming chat, an autonomous agent mode with tool use, Model Context Protocol (MCP) support, durable agent memory, local retrieval over the user's own files (RAG), a deep-research pipeline, a blind model-comparison arena, an AI-assisted writing editor, a hardware-aware model cookbook, and editor integration — while remaining radically neutral about *which* language model does the thinking. Every model backend, local or hosted, connects through a small provider-plugin contract; the rest of the application never knows which backend is talking. This paper describes the motivation, architecture, and security model of the system, and the design decisions that keep a one-person-maintainable codebase genuinely extensible.

1. Motivation

Interacting with large language models increasingly means renting access to a vertically integrated stack: one vendor's model, behind one vendor's app, with conversation history living on one vendor's servers. Projects such as PewDiePie's **Odysseus** (2026) demonstrated broad appetite for an alternative: a workspace you run yourself, where conversations, keys, and files never leave your machine.

Syrudas AI starts from the same local-first premise but elevates one idea to an organizing principle: **the model is a plugin**. Model backends appear and disappear monthly — a workspace should outlive all of them. Concretely, the goals are:

1. **Any model.** Local weights through Ollama or LM Studio, hosted frontier models through OpenRouter or first-party APIs — one UI, one history, one agent runtime across all of them.
2. **Local-first, private by construction.** The server binds to `127.0.0.1` only and rejects non-loopback Host headers. There is no telemetry, no account, and no cloud component. API keys are stored locally and masked in every API response.
3. **Agentic, but consent-gated.** The model can plan and act — run commands, read and write files, search and read the web, remember facts, search indexed documents — under an explicit, per-action human approval model for the dangerous parts.
4. **A hub, not a silo.** The workspace speaks the OpenAI wire dialect in both directions, so external tools (editors, scripts, other apps) can use the workspace's configured models as a service.
5. **Shippable.** A single portable executable a non-developer can unzip and double-click, with zero-configuration onboarding when a local model server is already running.

2. System overview

Syrudas AI is a two-tier application:

- **Backend:** Python 3.13 / FastAPI. Serves a REST + streaming API, owns the

SQLite database, runs the agent and research loops, and hosts the provider, MCP, and retrieval subsystems.

- **Frontend:** React 19 + TypeScript (Vite). A single-page interface —

chat, editor, arena, cookbook, settings — compiled to static assets and served by the backend, the same bundle in the browser, the desktop window, and the VS Code panel.

The packaged desktop app wraps both in a native window (pywebview over WebView2), with the server running on a background thread of the same process. State is deliberately boring: one SQLite file for conversations, messages, provider instances, MCP registrations, memories, indexed-document chunks, documents, arena results, and settings; one workspace folder for agent file output. In the portable build both live next to the executable, so copying the folder copies the installation.

```
+----- SyrudasAI.exe -----+
| pywebview window (WebView2) |
| +-- React SPA -- NDJSON/HTTP --> FastAPI (127.0.0.1:8040) |
|   chat · agent · research ·   +-- Host-guard middleware |
|   knowledge · arena · editor · +-- /api/chat · /api/research |
|   cookbook · settings         +-- /api/documents · /api/knowledge |
|                               +-- /api/arena · /api/cookbook |
|                               +-- /v1 (OpenAI-compatible hub) |
|                               +-- Agent loop -- builtin tools |
|                               | +----- MCP client --> studio servers |
|                               +-- Provider registry |
|                                   +-- openai_compat (builtin) |
|                                   +-- plugins/*.py (drop-in) |
|                               SQLite (syrudas.db) · data/workspace · plugins/ |
+-----+
| ^ | ^ | |
| | VS Code extension (panel, | Continue / any OpenAI- |
| | ask-about-selection) | compatible client via /v1 |
```

3. The provider abstraction

The core contract is one small class. A provider *type* is a Python class; a provider *instance* is that class configured with user data (URL, key) in the settings UI and stored in SQLite.

```
class ModelProvider(ABC):
    type_id: str # "openai_compat", "anthropic", ...
    display_name: str
    config_fields: list[ConfigField] # drives the settings form

    async def list_models(self) -> list[ModelInfo]: ...
    async def chat(self, model, messages, tools=None, params=None)
        -> AsyncIterator[StreamEvent]: ...
    async def embed(self, model, texts) -> list[list[float]]: ... # optional
```

Three design decisions matter here:

A normalized interior. Messages, tool specifications, tool calls, and stream events are defined once (`server/schemas.py`) in an OpenAI-flavored shape. Adapters translate at the edge; the chat pipeline, agent loop, persistence layer, and UI are all provider-agnostic. Adding Anthropic or Gemini support means writing one translation file, not touching the app.

Streaming as the only mode. `chat()` yields a flat event stream — `text_delta`, `tool_call`, `usage`, `error`, `done`. Non-streaming responses are just a stream that ends quickly, so there is exactly one code path.

Discovery by drop-in. The registry imports builtin adapters statically (a hard requirement under PylInstaller, whose bundles are invisible to `pkgutil` directory scanning) and additionally scans a user-writable `plugins/` folder next to the executable. A user can add a new backend to the *packaged* app by dropping a `.py` file beside it — no rebuild.

The single builtin adapter, `openai_compat`, covers most of the ecosystem in practice: Ollama, LM Studio, llama.cpp server, vLLM, OpenRouter, and OpenAI itself all speak the same dialect. Optional connectors for **Anthropic (Claude)** and **Google (Gemini)** ship as drop-in plugins. Embedding support is an optional fifth method: providers that implement `embed()` become eligible for the retrieval subsystem.

4. Chat pipeline

`POST /api/chat` accepts a message, persists it, and returns a streamed NDJSON body — one JSON event per line, consumed by the frontend with a `ReadableStream` reader. NDJSON-over-POST was chosen over WebSockets (no connection lifecycle to manage) and over EventSource (which cannot POST). Interleaved event types let a single stream carry plain chat, agent tool activity, and approval requests uniformly.

Messages are persisted as they complete, so a conversation survives interrupted streams, application restarts, and version upgrades. Titles derive from the first user message; attached-file blocks are stripped first. A per-conversation generation counter guards against zombie streams writing into a conversation that was rewound or deleted underneath them.

5. Agent mode

Agent mode turns the chat into a plan-act loop: the model receives tool schemas alongside the conversation; returned tool calls are executed and their results appended; the loop continues until the model stops calling tools or a step ceiling (15) is reached.

Builtin tools:

Tool	Capability	Guard
<code>shell</code>	PowerShell command	Per-call human approval
<code>file_read / file_list</code>	Text file I/O	Path sandbox
<code>file_write</code>	Write a text file	Free in workspace; approval outside it
<code>web_search</code>	DuckDuckGo search	—
<code>web_fetch</code>	URL → readable text	Per-call approval ; blocks private IPs
<code>memory_save / memory_delete / memory_search</code>	Durable facts	Ungated (user-visible)
<code>knowledge_search</code>	Retrieve indexed passages	Ungated (read-only)

The approval gate. The HTTP stream is one-way, so consent arrives out-of-band: when the agent proposes a gated call, the loop emits an `approval_required` event, parks on an `asyncio.Future`, and resumes only when the user clicks Approve or Deny in the UI (`POST /api/approvals/{id}`). A denial is reported to the model as an ordinary tool result, so it can adjust course rather than crash. Gating is a per-call decision — the `Tool.needs_approval(args)` hook lets `file_write` be free inside the workspace but require approval for a granted external folder.

The file sandbox. Relative paths resolve inside a dedicated workspace folder. Absolute paths are honored only inside folders the user has explicitly granted in Settings; everything else is refused with an error the model can read. Grants are stored as data, surfaced to the model in its system prompt, and revocable at any time.

Memory. `memory_save` records a short, distilled fact; the newest memories (within a character budget) are injected into the agent's system prompt at the start of each agent-mode conversation, so context carries across sessions. Memory is deliberately ungated: it has no effect outside the app, every save appears as a tool card, and Settings → Agent memory is an always-available review-and-delete surface. Plain (non-agent) chat never sees memories.

MCP. Users can register stdio MCP servers (command + arguments). Their tools are namespaced (`filesystem_read_file`) and merged into the agent's toolset. One implementation subtlety is worth recording: each server connection is owned by a dedicated `asyncio` task, because `anyio` cancel scopes must be entered and exited by the same task; other tasks only use the initialized session object.

6. Knowledge: local retrieval (RAG)

The knowledge subsystem lets the workspace answer from documents far larger than any context window, without those documents leaving the machine. Files or folders (text, code, PDFs) are indexed under Settings → Knowledge: each source is chunked (paragraph-aware, with overlap), embedded through a provider that implements `embed()`, and stored as normalized `float32` vectors in SQLite. Search embeds the query and ranks chunks by cosine similarity — a brute-force dot product in pure Python, which is milliseconds at the few-thousand-chunk scale this app targets, so there is no vector database and no numpy dependency.

Retrieval is exposed to the agent as the read-only `knowledge_search` tool and reused by the deep-research pipeline. Two properties keep it safe and honest: indexing is sandboxed to the same allowed roots as the file tools (re-resolving every walked file so a junction or symlink cannot escape), and changing the embedding model clears the index, because vectors from different models are not comparable.

7. Deep research

Deep Research is a deterministic **plan** → **search** → **read** → **synthesize** pipeline rather than an autonomous tool loop, so the outcome is predictable even with smaller local models. One completion turns the question into a few search queries; each query runs through web search and the candidate URLs are deduplicated; the top sources are fetched to readable text (SSRF-guarded) and the local knowledge index is searched too; a final streamed completion writes a cited Markdown report from the numbered sources, followed by a Sources list. Untrusted page text is fenced and marked as data in the synthesis prompt to

blunt instruction injection, and source titles are sanitized before they enter the report. The whole run is persisted as an ordinary conversation, so history, export, and the sidebar work with no extra machinery.

8. The writing editor

The editor is a local document workspace: a document list, a title, and a body that autosaves. Select text and apply a preset action (Improve, Shorten, Expand, Fix grammar), Continue from the cursor, or give a custom instruction; the AI suggestion streams into a panel to Accept or Reject. `POST /api/documents/edit` is a stateless streaming endpoint that returns only the replacement text — it never mutates a document server-side; the client decides whether to splice the accepted text at the captured selection. Autosave is a dirty-flag debounce backed by a stale-proof mirror that flushes on document switch and on leaving the view, and the editor is locked while a suggestion is pending so an accepted edit cannot land at drifted offsets.

9. The blind arena

The arena pits two chosen models against the same prompt with their names hidden — a coin flip decides which model fills which column, so position never leaks identity. Both answers stream in parallel via two stateless `/api/complete` calls; the user votes (A / B / tie / both-bad), which reveals the names and records the result to a local per-model win/loss/tie leaderboard. The multi-provider core does the routing; the arena is mostly a view on top of it.

10. The model cookbook

The cookbook detects local hardware (CPU, RAM, and GPU VRAM via `nvidia-smi` with a WMI fallback, all `stdlib`-only and degrading to unknowns rather than raising) and rates a curated catalog of models as *fits your GPU / tight / CPU / too big* for that machine. Models can be downloaded straight into Ollama with streamed pull progress, and removed again. It is strictly additive: the cookbook only helps get models into Ollama, after which they appear through the normal provider and model picker. Hardware detection runs off the event loop so it never freezes concurrent streams.

11. Interoperability

Syrudas is both a *client* of the OpenAI dialect (through `openai_compat`) and a *server* of it. The `/v1` surface exposes every model of every configured provider instance under a namespaced id (`ollama-local/llama3.1:8b`), with `chat/completions` translating each request to the normalized schema and routing it to the right backend — streaming, tool calls, and usage included. These calls are stateless and never touch conversation history.

This inverts the usual integration burden. Instead of Syrudas implementing an editor plugin for every tool, any OpenAI-compatible tool can adopt Syrudas as its backend by changing a base URL. The Continue extension for VS Code is the reference consumer.

A dedicated VS Code extension complements the hub: it embeds the full workspace UI in an editor panel and adds a right-click *Ask About Selection* command that prefills a chat with the selected code (file, line, language fence) via a `?prompt=` deep-link parameter.

12. File attachments

Attachments follow a deliberately stateless pipeline: the client uploads a file to `POST /api/attachments`, the server extracts text (UTF-8 for text-like formats; pypdf for PDFs; binaries rejected; 120k-character cap with explicit truncation flags), and returns it. The client embeds the text in the outgoing message inside `<file name="...">` delimiters.

Storing attachment content *in the message itself* — rather than in a side-table of blobs — buys three properties at once: history replay to any provider needs no special casing, export/backup is just the database file, and the UI can reconstruct file chips (collapsible, per-file) from the message text alone. For documents too large for the model's context window, the knowledge subsystem (Section 6) is the better path.

13. Security and privacy model

The threat model is honest about what a local, single-user tool can promise.

- **Network exposure:** the server binds to `127.0.0.1` exclusively, and a

middleware rejects any request whose `Host` header is not a loopback name. That closes the DNS-rebinding / drive-by-CSRF vector where a web page the user is visiting POSTs to the local server; there is otherwise no remote surface and thus no authentication layer.

- **Server-side request forgery:** the web-fetch path (used by the agent and

by deep research) refuses any URL that resolves to a private, loopback, or link-local address, re-checking on every redirect hop, so a fetched page cannot bounce the agent onto localhost, the app's own API, or the LAN.

- **Model-initiated actions:** the dangerous capabilities — arbitrary shell,

web fetch, and file writes outside the workspace — are gated per call with no persistent allow. File access is deny-by-default outside the workspace plus explicit grants. Injected instructions in fetched or retrieved text are treated as data, not commands.

- **Data at rest:** conversations, settings, memories, indexed chunks,

documents, and provider API keys live in a local SQLite file. Keys are stored in plaintext (an OS keychain integration is future work) but are masked (`...1234`) in every API response, so the browser/UI layer never re-receives full secrets. Anyone with read access to the data folder can read the keys; treat it accordingly.

- **Telemetry:** none. The application makes outbound connections only to

model backends the user configured and URLs the user (or their agent, under the rules above) requests.

- **Residual risks:** a granted folder is fully readable/writable by any

model the user runs, including a badly aligned local one; MCP servers run with user privileges and are trusted by registration; the unsigned executable requires a SmartScreen click-through. These are documented rather than hidden.

14. Themes and accessibility

Appearance and colour vision are two independent axes, so a colour-blind user can still choose light or dark. Appearance (system / light / dark) and colour vision (default, protanopia, deuteranopia, tritanopia, achromatopsia) are applied as attributes on the document root and resolved entirely through CSS variable overrides; a pre-render inline script applies them before first paint. Colour-vision modes remap only the four status hues and are tuned per background so status text meets WCAG AA contrast on both light and dark; achromatopsia drops hue entirely and relies on luminance. Because status is never conveyed by colour alone — every state also carries an icon and a text label — meaning survives any palette. The app also honours `prefers-reduced-motion`.

15. Packaging and distribution

The release artifact is a portable zip: one PyInstaller onefile executable (windowed, WebView2), the optional provider connectors, an end-user README, the setup guide, and the MIT license. Everything mutable lives beside the executable, making installations trivially copyable, movable, and deletable. On first run with an empty database, the server probes well-known local backends (Ollama on `:11434`, LM Studio on `:1234`) and auto-configures any that respond — a fresh user with a local model server running reaches a working chat with zero configuration steps.

Version identity is stamped from one source of truth (`APP_VERSION`): the health endpoint, the UI footer, and the Windows file properties of the executable. Two Windows-specific build lessons are preserved in the codebase for posterity: PyInstaller bundles are invisible to `pkgutil` (hence static builtin imports), and PowerShell 5.1 under `$ErrorActionPreference = "Stop"` converts native `stderr` log lines into terminating errors (hence `cmd /c "... 2>&1"` wrappers in the build scripts).

Every subsystem ships with an offline test suite that drives the real code paths against fakes — a scripted fake provider for the agent and research loops, `httpx.MockTransport` for the connectors and the Ollama client, a deterministic fake embedder for retrieval — so correctness is checked without a network, a GPU, or a running model.

16. Limitations and roadmap

Current limitations: single-user by design; Windows-only packaging (the server itself is portable Python); text-only multimodality; plaintext key storage; model *download* management is Ollama-specific.

Much of the original roadmap has since shipped: native Anthropic and Gemini connectors, deep research, the blind arena, retrieval over local files, and an AI writing editor are all present as of this version, alongside agent memory, the hardware cookbook, and the accessibility/theming work.

Planned directions, in rough order of value: image (vision) input through the normalized schema; an optional local token on the `/v1` hub for shared machines; OS-keychain key storage; and — the largest remaining gap relative to a full personal-assistant suite — productivity integrations such as email, calendar, and notes, each of which is effectively its own subsystem.

17. Conclusion

Syrudas AI demonstrates that a genuinely useful AI workspace — streaming chat, consent-gated agency, local retrieval, deep research, model comparison, a writing editor, protocol interoperability, editor integration, and one-click distribution — fits in a small, single-maintainer codebase when one abstraction is

chosen carefully and enforced everywhere. Models will keep changing. A workspace whose only opinion about models is a small provider contract is positioned to outlast any of them.

Syrudas AI is open source under the MIT license. Architecture reference: `README.md` and `docs/SETUP.md` in the repository.